

Assignment 5 Written Solutions

1. Attention exploration (20 points)

Multi-head self-attention is the core modeling component of Transformers. In this question, we'll get some practice working with the self-attention equations, and motivate why multi-headed self-attention can be preferable to single-headed self-attention.

Recall that attention can be viewed as an operation on a *query* vector $q \in \mathbb{R}^d$, a set of *value* vectors $\{v_1, \dots, v_n\}, v_i \in \mathbb{R}^d$, and a set of *key* vectors $\{k_1, \dots, k_n\}, k_i \in \mathbb{R}^d$, specified as follows:

$$c = \sum_{i=1}^n v_i \alpha_i \quad (1)$$

$$\alpha_i = \frac{\exp(k_i^\top q)}{\sum_{j=1}^n \exp(k_j^\top q)} \quad (2)$$

with $\alpha = \{\alpha_1, \dots, \alpha_n\}$ termed the “attention weights”. Observe that the output $c \in \mathbb{R}^d$ is an average over the value vectors weighted with respect to α .

- (a) (5 points) **Copying in attention.** One advantage of attention is that it's particularly easy to “copy” a value vector to the output c . In this problem, we'll motivate why this is the case.
- (1 point) **Explain** why α can be interpreted as a **categorical probability distribution**.
 - (2 points) The distribution α is typically relatively “diffuse”; the probability mass is spread out between many different α_i . However, this is not always the case. **Describe** (in one sentence) under what conditions the categorical distribution α puts almost all of its weight on some α_j , where $j \in \{1, \dots, n\}$ (i.e. $\alpha_j \gg \sum_{i \neq j} \alpha_i$). What must be true about the query q and/or the keys $\{k_1, \dots, k_n\}$?
 - (1 point) Under the conditions you gave in (ii), **describe** the output c .
 - (1 point) **Explain** (in two sentences or fewer) what your answer to (ii) and (iii) means intuitively. We are looking for what c looks like in terms of the value vectors $\{v_1, \dots, v_n\}$ based on the relation between q and the keys $\{k_1, \dots, k_n\}$.

- (i) In probability theory and statistics, a categorical distribution is a discrete probability distribution that describes the possible results of a random variable that can take on one of n possible categories, with the probability of each category separately specified. The parameters specifying the probabilities of each possible outcome are constrained only by the fact that each must be in the range 0 to 1, and all must sum to 1.

$$\text{In attention, } \alpha_i = \frac{\exp(k_i^\top q)}{\sum_{j=1}^n \exp(k_j^\top q)} < 1$$

$$\text{and } \sum_{i=1}^n \alpha_i = \sum_{i=1}^n \frac{\exp(k_i^\top q)}{\sum_{j=1}^n \exp(k_j^\top q)} = 1$$

Consequently, α_i can be interpreted as a categorical probability distribution.

- (ii) $\alpha_j \gg \sum_{i \neq j} \alpha_i$ when the numerator of α_j is very large or the denominator is small. Since the denominator is the same for all α , the condition is satisfied when the numerator,

$\exp(k_j^T q)$, gets very large. This occurs when $k_j \approx q$, which means that the i^{th} key matches the query.

(iii) We can write $c = \sum_{i \neq j}^n v_i \alpha_i + v_j \alpha_j$

$$\because \alpha_j \gg \sum_{i \neq j} \alpha_i$$

$$\therefore v_j \alpha_j \gg \sum_{i \neq j}^n v_i \alpha_i$$

$$\therefore c \cong v_j \alpha_j$$

(iv) Since attention is basically a look-up table (with the query matches all the keys with a categorical probability distribution), it is very logical that when a certain key k_i matches the query q with great probability, the output is the value v_i produced from that key k_i .

(b) (7 points) **An average of two.** Instead of focusing on just one vector v_j , a Transformer model might want to incorporate information from *multiple* source vectors. Consider the case where we instead want to incorporate information from **two** vectors v_a and v_b , with corresponding key vectors k_a and k_b .

- i. (3 points) How should we combine two d -dimensional vectors v_a, v_b into one output vector c in a way that preserves information from both vectors? In machine learning, one common way to do so is to take the average: $c = \frac{1}{2}(v_a + v_b)$. It might seem hard to extract information about the original vectors v_a and v_b from the resulting c , but under certain conditions one can do so. In this problem, we'll see why this is the case.

Suppose that although we don't know v_a or v_b , we do know that v_a lies in a subspace A formed by the m basis vectors $\{a_1, a_2, \dots, a_m\}$, while v_b lies in a subspace B formed by the p basis vectors $\{b_1, b_2, \dots, b_p\}$. (This means that any v_a can be expressed as a linear combination of its basis vectors, as can v_b . All basis vectors have norm 1 and are orthogonal to each other.)

Additionally, suppose that the two subspaces are orthogonal; i.e. $a_j^\top b_k = 0$ for all j, k .

Using the basis vectors $\{a_1, a_2, \dots, a_m\}$, construct a matrix M such that for arbitrary vectors $v_a \in A$ and $v_b \in B$, we can use M to extract v_a from the sum vector $s = v_a + v_b$. In other words, we want to construct M such that for any v_a, v_b , $Ms = v_a$. Show that $Ms = v_a$ holds for your M .

Note: There are several ways to approach this problem. A hint that can be useful for one approach: given that the vectors $\{a_1, a_2, \dots, a_m\}$ are both *orthogonal* and *form a basis* for v_a , we know that there exist some $c_1, c_2, \dots, c_m$ such that $v_a = c_1 a_1 + c_2 a_2 + \dots + c_m a_m$. Can you create a vector of these weights c ?

- ii. (4 points) As before, let v_a and v_b be two value vectors corresponding to key vectors k_a and k_b , respectively. Assume that (1) all key vectors are orthogonal, so $k_i^\top k_j = 0$ for all $i \neq j$; and (2) all key vectors have norm 1.¹ **Find an expression** for a query vector q such that $c \approx \frac{1}{2}(v_a + v_b)$, and justify your answer.² (Recall what you learned in part (a).)

(i) $v_a = c_1 a_1 + c_2 a_2 + \dots + c_m a_m = \bar{A}c$ where $a_i \in \mathbb{R}^{d \times 1}$, $\bar{A} \in \mathbb{R}^{d \times m}$, $c \in \mathbb{R}^{m \times 1}$, and $v_a \in \mathbb{R}^{d \times 1}$
 $v_b = f_1 b_1 + f_2 b_2 + \dots + f_p b_p = \bar{B}f$ where $b_i \in \mathbb{R}^{d \times 1}$, $\bar{B} \in \mathbb{R}^{d \times p}$, $f \in \mathbb{R}^{p \times 1}$, and $v_b \in \mathbb{R}^{d \times 1}$

$$s = v_a + v_b \in \mathbb{R}^{d \times 1}$$

$$\therefore Ms = v_a, s = v_a + v_b$$

$$\therefore Mv_b = 0, M \in \mathbb{R}^{d \times d}$$

$$\therefore (1) M\bar{A}c = \bar{A}c, (2) M\bar{B}f = 0$$

For (1): $\because \{a_1, a_2, \dots, a_m\}$ are orthonormal

$$\therefore a_i^\top a_j = 0 \text{ where } i \neq j$$

$$, a_i^\top a_j = 1 \text{ where } i = j$$

$$\therefore M = \bar{A}^\top \Rightarrow \bar{A}^\top \bar{A}c = Ic = c \text{ where } c \text{ is the representation of } v_a \text{ in vector form}$$

For (2): $\because a_j^\top b_k = 0$ for all j, k

$$\therefore \bar{A}^\top \bar{B}f = 0$$

$$\therefore M = \bar{A}^\top \text{ holds true for both (1) and (2)}$$

$$(ii) \quad c = \sum_{i=1}^n v_i \alpha_i = \sum_{i=1}^n v_i \frac{\exp(k_i^T q)}{\sum_{j=1}^n \exp(k_j^T q)} = \frac{v_1 \exp(k_1^T q) + v_2 \exp(k_2^T q) + \dots + v_n \exp(k_n^T q)}{\exp(k_1^T q) + \exp(k_2^T q) + \dots + \exp(k_n^T q)}$$

$$c \approx \frac{1}{2} (v_a + v_b) \Rightarrow \alpha_a = \alpha_b = \frac{1}{2}$$

which means $k_a^T q = k_b^T q \gg k_i^T q$ for $i \neq a, b$

Let $k_a^T q = k_b^T q = \gamma$

$$\alpha_a = \frac{\exp(k_a^T q)}{\sum_{j=1}^n \exp(k_j^T q)} = \frac{\exp(\gamma)}{\exp(k_a^T q) + \exp(k_b^T q) + \exp(k_3^T q) + \dots + \exp(k_n^T q)} = \frac{\exp(\gamma)}{2 \exp(\gamma) + \exp(k_3^T q) + \dots + \exp(k_n^T q)}$$

But $k_a^T q = k_b^T q \gg k_i^T q$ for $i \neq a, b \Rightarrow k_i^T q \approx 0$

$$\alpha_a = \frac{\exp(\gamma)}{2 \exp(\gamma) + \exp(0) + \dots + \exp(0)} = \frac{\exp(\gamma)}{2 \exp(\gamma) + n - 2}$$

For $\gamma \gg 0$: $\exp(\gamma) \rightarrow \infty \Rightarrow 2 \exp(\gamma) + n - 2 \approx 2 \exp(\gamma)$

$$\alpha_a = \frac{\exp(\gamma)}{2 \exp(\gamma)} = \frac{1}{2}$$

$$\therefore k_a^T q = k_b^T q = \gamma, k_a^T k_a = k_b^T k_b = 1, k_a^T k_b = k_b^T k_a = 0$$

$$\therefore q = \gamma(k_a + k_b) \text{ with } \gamma \gg 0$$

(c) (5 points) **Drawbacks of single-headed attention:** In the previous part, we saw how it was *possible* for a single-headed attention to focus equally on two values. The same concept could easily be extended to any subset of values. In this question we'll see why it's not a *practical* solution. Consider a set of key vectors $\{k_1, \dots, k_n\}$ that are now randomly sampled, $k_i \sim \mathcal{N}(\mu_i, \Sigma_i)$, where the means $\mu_i \in \mathbb{R}^d$ are known to you, but the covariances Σ_i are unknown. Further, assume that the means μ_i are all perpendicular; $\mu_i^\top \mu_j = 0$ if $i \neq j$, and unit norm, $\|\mu_i\| = 1$.

- i. (2 points) Assume that the covariance matrices are $\Sigma_i = \alpha I, \forall i \in \{1, 2, \dots, n\}$, for vanishingly small α . Design a query q in terms of the μ_i such that as before, $c \approx \frac{1}{2}(v_a + v_b)$, and provide a brief argument as to why it works.
- ii. (3 points) Though single-headed attention is resistant to small perturbations in the keys, some types of larger perturbations may pose a bigger issue. Specifically, in some cases, one key vector k_a may be larger or smaller in norm than the others, while still pointing in the same direction as μ_a . As an example, let us consider a covariance for item a as $\Sigma_a = \beta I + \frac{1}{2}(\mu_a \mu_a^\top)$ for vanishingly small β (as shown in figure 1). This causes k_a to point in roughly the same direction as μ_a , but with large variances in magnitude. Further, let $\Sigma_i = \beta I$ for all $i \neq a$.

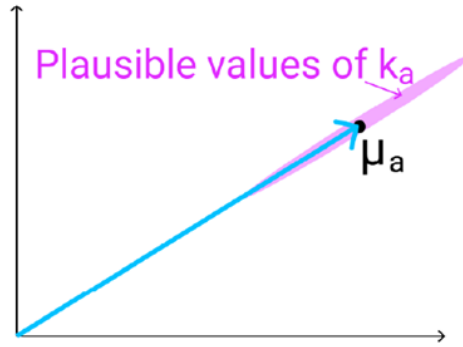


Figure 1: The vector μ_a (shown here in 2D as an example), with the range of possible values of k_a shown in red. As mentioned previously, k_a points in roughly the same direction as μ_a , but may have larger or smaller magnitude.

When you sample $\{k_1, \dots, k_n\}$ multiple times, and use the q vector that you defined in part i., what do you expect the vector c will look like qualitatively for different samples? Think about how it differs from part (i) and how c 's variance would be affected.

- (i) From (b) we know that $q = \gamma(k_a + k_b)$ with $\gamma \gg 0$

$$\text{Let } k_a = \mu_a \pm \Sigma_a \mu_a = \mu_a \pm \alpha_a I \mu_a = \mu_a \pm \alpha_a \mu_a = (1 \pm \alpha_a) \mu_a, \quad k_b = (1 \pm \alpha_b) \mu_b$$

$$\therefore q = \gamma((1 \pm \alpha_a) \mu_a + (1 \pm \alpha_b) \mu_b)$$

$$\therefore \alpha_a, \alpha_b \text{ are vanishingly small}$$

$$\therefore (1 \pm \alpha_a) \approx 1, \quad (1 \pm \alpha_b) \approx 1$$

$$\therefore q = \gamma(\mu_a + \mu_b)$$

- (ii) $k_a = \mu_a \pm \Sigma_a \mu_a = \mu_a \pm \left(\beta I + \frac{1}{2}(\mu_a \mu_a^\top) \right) \mu_a = \mu_a \pm \beta I \mu_a \pm \frac{1}{2} \mu_a (\mu_a^\top \mu_a) = \mu_a \pm \beta \mu_a \pm \frac{1}{2} \mu_a$

$$k_a = \left(1 \pm \beta \pm \frac{1}{2} \right) \mu_a$$

$\therefore \beta$ is vanishingly small

$$\therefore \left(1 \pm \beta \pm \frac{1}{2}\right) \approx \left(1 \pm \frac{1}{2}\right)$$

$$\therefore k_a = \left(1 \pm \frac{1}{2}\right) \mu_a$$

For $i \neq a$: $k_i = \mu_i + \Sigma_i \mu_i = \mu_i \pm \beta I \mu_i = (1 \pm \beta) \mu_i$

$\therefore \beta$ is vanishingly small

$$\therefore (1 \pm \beta) \approx 1$$

$$\therefore k_i = \mu_i$$

$$\therefore q = \gamma(k_a + k_b) = \gamma \left(\left(1 \pm \frac{1}{2}\right) \mu_a + \mu_b \right) \text{ with } \gamma \gg 0$$

$$c = \sum_{i=1}^n v_i \alpha_i = \sum_{i=1}^n v_i \frac{\exp(k_i^T q)}{\sum_{j=1}^n \exp(k_j^T q)}$$

$$c = \frac{v_a \exp(k_a^T q) + v_b \exp(k_b^T q) + v_3 \exp(k_3^T q) + \dots + v_n \exp(k_n^T q)}{\exp(k_a^T q) + \exp(k_b^T q) + \exp(k_3^T q) + \dots + \exp(k_n^T q)}$$

$$c = \frac{v_a \exp\left(\mu_a^T \gamma \left(\left(1 \pm \frac{1}{2}\right) \mu_a + \mu_b \right)\right) + v_b \exp\left(\mu_b^T \gamma \left(\left(1 \pm \frac{1}{2}\right) \mu_a + \mu_b \right)\right) + v_3 \exp\left(\mu_3^T \gamma \left(\left(1 \pm \frac{1}{2}\right) \mu_a + \mu_b \right)\right) + \dots + v_n \exp\left(\mu_n^T \gamma \left(\left(1 \pm \frac{1}{2}\right) \mu_a + \mu_b \right)\right)}{\exp\left(\mu_a^T \gamma \left(\left(1 \pm \frac{1}{2}\right) \mu_a + \mu_b \right)\right) + \exp\left(\mu_b^T \gamma \left(\left(1 \pm \frac{1}{2}\right) \mu_a + \mu_b \right)\right) + \exp\left(\mu_3^T \gamma \left(\left(1 \pm \frac{1}{2}\right) \mu_a + \mu_b \right)\right) + \dots + \exp\left(\mu_n^T \gamma \left(\left(1 \pm \frac{1}{2}\right) \mu_a + \mu_b \right)\right)}$$

$$c = \frac{v_a \exp\left(\gamma \left(1 \pm \frac{1}{2}\right)\right) + v_b \exp(\gamma) + v_3 + \dots + v_n}{\exp\left(\gamma \left(1 \pm \frac{1}{2}\right)\right) + \exp(\gamma) + 1 + \dots + 1} = \frac{v_a \exp\left(\gamma \left(1 \pm \frac{1}{2}\right)\right) + v_b \exp(\gamma) + v_3 + \dots + v_n}{\exp\left(\gamma \left(1 \pm \frac{1}{2}\right)\right) + \exp(\gamma) + n - 2}$$

$$\alpha_a = \frac{\exp\left(\gamma \left(1 \pm \frac{1}{2}\right)\right)}{\exp\left(\gamma \left(1 \pm \frac{1}{2}\right)\right) + \exp(\gamma) + n - 2} \approx \frac{\exp\left(\gamma \left(1 \pm \frac{1}{2}\right)\right)}{\exp\left(\gamma \left(1 \pm \frac{1}{2}\right)\right) + \exp(\gamma)} = \frac{1}{1 + \exp\left(-\gamma \left(1 \pm \frac{1}{2} - 1\right)\right)} = \frac{1}{1 + \exp\left(-\gamma \left(\pm \frac{1}{2}\right)\right)}$$

$$\alpha_b = \frac{\exp(\gamma)}{\exp\left(\gamma \left(1 \pm \frac{1}{2}\right)\right) + \exp(\gamma) + n - 2} \approx \frac{\exp(\gamma)}{\exp\left(\gamma \left(1 \pm \frac{1}{2}\right)\right) + \exp(\gamma)} = \frac{1}{1 + \exp\left(\gamma \left(1 \pm \frac{1}{2} - 1\right)\right)} = \frac{1}{1 + \exp\left(\gamma \left(\pm \frac{1}{2}\right)\right)}$$

$$\alpha_i = \frac{1}{\exp\left(\gamma \left(1 \pm \frac{1}{2}\right)\right) + \exp(\gamma) + n - 2} \approx \frac{1}{\exp\left(\gamma \left(1 \pm \frac{1}{2}\right)\right) + \exp(\gamma)} \text{ for } i \neq a, b$$

For variance of $+\frac{1}{2}$ and a large $\gamma \gg 0$:

$$\alpha_a = \frac{1}{1 + \exp(-\infty)} = 1, \alpha_b = \frac{1}{1 + \exp(\infty)} = 0, \alpha_i = \frac{1}{\exp(\infty) + \exp(\infty)} = 0$$

$$\therefore c = v_a$$

For variance of $-\frac{1}{2}$ and a large $\gamma \gg 0$:

$$\alpha_a = \frac{1}{1 + \exp(\infty)} = 0, \alpha_b = \frac{1}{1 + \exp(-\infty)} = 1, \alpha_i = \frac{1}{\exp(\infty) + \exp(\infty)} = 0$$

$$\therefore c = v_b$$

The value of c will oscillate between two values v_a and v_b even with the small value of variance $\pm \frac{1}{2}$ between the keys

(d) (3 points) **Benefits of multi-headed attention:** Now we'll see some of the power of multi-headed attention. We'll consider a simple version of multi-headed attention which is identical to single-headed self-attention as we've presented it in this homework, except two query vectors (q_1 and q_2) are defined, which leads to a pair of vectors (c_1 and c_2), each the output of single-headed attention given its respective query vector. The final output of the multi-headed attention is their average, $\frac{1}{2}(c_1 + c_2)$. As in question 1(c), consider a set of key vectors $\{k_1, \dots, k_n\}$ that are randomly sampled, $k_i \sim \mathcal{N}(\mu_i, \Sigma_i)$, where the means μ_i are known to you, but the covariances Σ_i are unknown. Also as before, assume that the means μ_i are mutually orthogonal; $\mu_i^\top \mu_j = 0$ if $i \neq j$, and unit norm, $\|\mu_i\| = 1$.

- i. (1 point) Assume that the covariance matrices are $\Sigma_i = \alpha I$, for vanishingly small α . Design q_1 and q_2 in terms of μ_i such that c is approximately equal to $\frac{1}{2}(v_a + v_b)$. Note that q_1 and q_2 should have different expressions.
- ii. (2 points) Assume that the covariance matrices are $\Sigma_a = \alpha I + \frac{1}{2}(\mu_a \mu_a^\top)$ for vanishingly small α , and $\Sigma_i = \alpha I$ for all $i \neq a$. Take the query vectors q_1 and q_2 that you designed in part i. What, qualitatively, do you expect the output c to look like across different samples of the key vectors? Explain briefly in terms of variance in c_1 and c_2 . You can ignore cases in which $k_a^\top q_i < 0$.

(i) $c = \frac{1}{2}(v_a + v_b) = \frac{1}{2}(c_1 + c_2)$

$$c_1 = v_a, c_2 = v_b$$

For c_1, α_a must be 1:

$$\alpha_a = \frac{\exp(k_a^\top q_1)}{\exp(k_a^\top q_1) + \exp(k_b^\top q_1) + \exp(k_3^\top q_1) + \dots + \exp(k_n^\top q_1)} = 1$$

$$\text{Which means } \exp(k_b^\top q_1) = \exp(k_3^\top q_1) = \dots = \exp(k_n^\top q_1) = 0$$

$$\therefore q_1 = k_a$$

We know from (b) that $k_a = \mu_a$

$$\therefore q_1 = \mu_a$$

For c_2, α_b must be 1:

$$\alpha_b = \frac{\exp(k_b^\top q_2)}{\exp(k_a^\top q_2) + \exp(k_b^\top q_2) + \exp(k_3^\top q_2) + \dots + \exp(k_n^\top q_2)} = 1$$

$$\text{Which means } \exp(k_a^\top q_2) = \exp(k_3^\top q_2) = \dots = \exp(k_n^\top q_2) = 0$$

$$\therefore q_2 = k_b$$

We know from (b) that $k_b = \mu_b$

$$\therefore q_2 = \mu_b$$

(ii) From (c): $k_a = \left(1 \pm \frac{1}{2}\right) \mu_a, k_i = \mu_i$

$$q_1 = k_a = \left(1 \pm \frac{1}{2}\right) \mu_a, q_2 = k_b = \mu_b$$

$$c_1 = \sum_{i=1}^n v_i \alpha_i = \sum_{i=1}^n v_i \frac{\exp(k_i^\top q_1)}{\sum_{j=1}^n \exp(k_j^\top q_1)}$$

$$c_1 = \frac{v_a \exp(k_a^\top q_1) + v_b \exp(k_b^\top q_1) + v_3 \exp(k_3^\top q_1) + \dots + v_n \exp(k_n^\top q_1)}{\exp(k_a^\top q_1) + \exp(k_b^\top q_1) + \exp(k_3^\top q_1) + \dots + \exp(k_n^\top q_1)}$$

$$\alpha_a = \frac{\exp(k_a^T q_1)}{\exp(k_a^T q_1) + \exp(k_b^T q_1) + \exp(k_3^T q_1) + \dots + \exp(k_n^T q_1)}$$

$$\alpha_a = \frac{\exp\left(\left(1 \pm \frac{1}{2}\right) \mu_a^T \left(1 \pm \frac{1}{2}\right) \mu_a\right)}{\exp\left(\left(1 \pm \frac{1}{2}\right) \mu_a^T \left(1 \pm \frac{1}{2}\right) \mu_a\right) + \exp\left(\mu_b^T \left(1 \pm \frac{1}{2}\right) \mu_a\right) + \exp\left(\mu_3^T \left(1 \pm \frac{1}{2}\right) \mu_a\right) + \dots + \exp\left(\mu_n^T \left(1 \pm \frac{1}{2}\right) \mu_a\right)}$$

$$\alpha_a = \frac{\exp\left(\left(1 \pm \frac{1}{2}\right) \left(1 \pm \frac{1}{2}\right)\right)}{\exp\left(\left(1 \pm \frac{1}{2}\right) \left(1 \pm \frac{1}{2}\right)\right) + 1 + \dots + 1} = \frac{\exp\left(\left(1 \pm \frac{1}{2}\right) \left(1 \pm \frac{1}{2}\right)\right)}{\exp\left(\left(1 \pm \frac{1}{2}\right) \left(1 \pm \frac{1}{2}\right)\right) + n - 1}$$

$$\alpha_a = \begin{cases} \frac{\exp\left(\frac{3}{2} \times \frac{3}{2}\right)}{\exp\left(\frac{3}{2} \times \frac{3}{2}\right) + n - 1} = \begin{cases} \frac{9.4877}{9.4877 + n - 1}, & \text{positive variance} \\ \frac{1.284}{1.284 + n - 1}, & \text{negative variance} \end{cases} \\ \frac{\exp\left(\frac{1}{2} \times \frac{1}{2}\right)}{\exp\left(\frac{1}{2} \times \frac{1}{2}\right) + n - 1} \end{cases}$$

$$\alpha_i = \frac{\exp(k_i^T q_1)}{\exp(k_a^T q_1) + \exp(k_b^T q_1) + \exp(k_3^T q_1) + \dots + \exp(k_n^T q_1)}$$

$$\alpha_i = \frac{\exp\left(\mu_i^T \left(1 \pm \frac{1}{2}\right) \mu_a\right)}{\exp\left(\left(1 \pm \frac{1}{2}\right) \mu_a^T \left(1 \pm \frac{1}{2}\right) \mu_a\right) + \exp\left(\mu_b^T \left(1 \pm \frac{1}{2}\right) \mu_a\right) + \exp\left(\mu_3^T \left(1 \pm \frac{1}{2}\right) \mu_a\right) + \dots + \exp\left(\mu_n^T \left(1 \pm \frac{1}{2}\right) \mu_a\right)}$$

$$\alpha_i = \frac{1}{\exp\left(\left(1 \pm \frac{1}{2}\right) \left(1 \pm \frac{1}{2}\right)\right) + 1 + \dots + 1} = \frac{1}{\exp\left(\left(1 \pm \frac{1}{2}\right) \left(1 \pm \frac{1}{2}\right)\right) + n - 1}$$

$$\alpha_i = \begin{cases} \frac{1}{\exp\left(\frac{3}{2} \times \frac{3}{2}\right) + n - 1} = \begin{cases} \frac{1}{9.4877 + n - 1}, & \text{positive variance} \\ \frac{1}{1.284 + n - 1}, & \text{negative variance} \end{cases} \\ \frac{1}{\exp\left(\frac{1}{2} \times \frac{1}{2}\right) + n - 1} \end{cases}$$

It is clear that $\alpha_a \gg \alpha_i$ for any positive or negative variance: $c_1 \approx v_a$

$$c_2 = \sum_{i=1}^n v_i \alpha_i = \sum_{i=1}^n v_i \frac{\exp(k_i^T q_2)}{\sum_{j=1}^n \exp(k_j^T q_2)}$$

$$c_2 = \frac{v_a \exp(k_a^T q_2) + v_b \exp(k_b^T q_2) + v_3 \exp(k_3^T q_2) + \dots + v_n \exp(k_n^T q_2)}{\exp(k_a^T q_2) + \exp(k_b^T q_2) + \exp(k_3^T q_2) + \dots + \exp(k_n^T q_2)}$$

$$\alpha_b = \frac{\exp(k_b^T q_2)}{\exp(k_a^T q_2) + \exp(k_b^T q_2) + \exp(k_3^T q_2) + \dots + \exp(k_n^T q_2)}$$

$$\alpha_b = \frac{\exp(\mu_b^T \mu_b)}{\exp(\mu_a^T \mu_b) + \exp(\mu_b^T \mu_b) + \exp(k_3^T \mu_b) + \dots + \exp(k_n^T \mu_b)} = \frac{\exp(1)}{1 + \exp(1) + 1 + \dots + 1} = \frac{\exp(1)}{\exp(1) + n - 1} = \frac{2.7183}{2.7183 + n - 1}$$

$$\alpha_i = \frac{\exp(\mu_i^T \mu_b)}{\exp(\mu_a^T \mu_b) + \exp(\mu_b^T \mu_b) + \exp(k_3^T \mu_b) + \dots + \exp(k_n^T \mu_b)} = \frac{1}{1 + \exp(1) + 1 + \dots + 1} = \frac{1}{\exp(1) + n - 1} = \frac{1}{2.7183 + n - 1}$$

It is clear that $\alpha_a \gg \alpha_i$ and doesn't depend on variance: $c_2 \approx v_b$

$$\therefore c = \frac{1}{2}(c_1 + c_2) \approx \frac{1}{2}(v_a + v_b)$$

This time, the output c does not oscillate between the values because of variance

2. Pretrained Transformer models and knowledge access (35 points)

(d) (5 points) **Make predictions (without pretraining).**

Train your model on `birth_places_train.tsv`, and evaluate on `birth_dev.tsv`. Specifically, you should now be able to run the following three commands:

```
# Train on the names dataset
python src/run.py finetune vanilla wiki.txt \
    --writing_params_path vanilla.model.params \
    --finetune_corpus_path birth_places_train.tsv

# Evaluate on the dev set, writing out predictions
python src/run.py evaluate vanilla wiki.txt \
    --reading_params_path vanilla.model.params \
    --eval_corpus_path birth_dev.tsv \
    --outputs_path vanilla.nopretrain.dev.predictions

# Evaluate on the test set, writing out predictions
python src/run.py evaluate vanilla wiki.txt \
    --reading_params_path vanilla.model.params \
    --eval_corpus_path birth_test_inputs.tsv \
    --outputs_path vanilla.nopretrain.test.predictions
```

Training will take less than 10 minutes (on GCP). Report your model's accuracy on the dev set (as printed by the second command above). Similar to assignment 4, we also have Tensorboard logging in assignment 5 for debugging. It can be launched using `tensorboard --logdir expt/`. Don't be surprised if it is well below 10%; we will be digging into why in Part 3. As a reference point, we want to also calculate the accuracy the model would have achieved if it had just predicted "London" as the birth place for everyone in the dev set. Fill in `london_baseline.py` to calculate the accuracy of that approach and report your result in your write-up. You should be able to leverage existing code such that the file is only a few lines long.

The model's accuracy on the dev set: **1.8%**

The accuracy of predicting only "London" on the dev set: **5.0%**

(f) (10 points) **Pretrain, finetune, and make predictions. Budget 2 hours for training.**

Now fill in the *pretrain* portion of *run.py*, which will pretrain a model on the span corruption task. Additionally, modify your *finetune* portion to handle finetuning in the case *with* pretraining. In particular, if a path to a pretrained model is provided in the bash command, load this model before finetuning it on the birthplace prediction task. Pretrain your model on *wiki.txt* (which should take approximately two hours), finetune it on *NameDataset* and evaluate it. Specifically, you should be able to run the following four commands: (Don't be concerned if the loss appears to plateau in the middle of pretraining; it will eventually go back down.)

```
# Pretrain the model
python src/run.py pretrain vanilla wiki.txt \
    --writing_params_path vanilla.pretrain.params

# Finetune the model
python src/run.py finetune vanilla wiki.txt \
    --reading_params_path vanilla.pretrain.params \
    --writing_params_path vanilla.finetune.params \
    --finetune_corpus_path birth_places_train.tsv

# Evaluate on the dev set; write to disk
python src/run.py evaluate vanilla wiki.txt \
    --reading_params_path vanilla.finetune.params \
    --eval_corpus_path birth_dev.tsv \
    --outputs_path vanilla.pretrain.dev.predictions

# Evaluate on the test set; write to disk
python src/run.py evaluate vanilla wiki.txt \
    --reading_params_path vanilla.finetune.params \
    --eval_corpus_path birth_test_inputs.tsv \
    --outputs_path vanilla.pretrain.test.predictions
```

Report the accuracy on the dev set (printed by the third command above). We expect the dev accuracy will be at least 10%, and will expect a similar accuracy on the held out test set.

The accuracy of the model on the dev set: **23.2%**

(g) (10 points) **Research!** Write and try out a more efficient variant of Attention (Budget 2 hours for pretraining!)

```
# Pretrain the model
python src/run.py pretrain perceiver wiki.txt --bottleneck_dim 64 \
    --pretrain_lr 6e-3 --writing_params_path perceiver.pretrain.params

# Finetune the model
python src/run.py finetune perceiver wiki.txt --bottleneck_dim 64 \
    --reading_params_path perceiver.pretrain.params \
    --writing_params_path perceiver.finetune.params \
    --finetune_corpus_path birth_places_train.tsv

# Evaluate on the dev set; write to disk
python src/run.py evaluate perceiver wiki.txt --bottleneck_dim 64 \
    --reading_params_path perceiver.finetune.params \
    --eval_corpus_path birth_dev.tsv \
    --outputs_path perceiver.pretrain.dev.predictions

# Evaluate on the test set; write to disk
python src/run.py evaluate perceiver wiki.txt --bottleneck_dim 64 \
    --reading_params_path perceiver.finetune.params \
    --eval_corpus_path birth_test_inputs.tsv \
    --outputs_path perceiver.pretrain.test.predictions
```

Report the accuracy of your perceiver attention model on birthplace prediction on birth_dev.tsv after pretraining and fine-tuning.

Save the predictions of the model on birth_test_inputs.tsv to perceiver.pretrain.test.predictions

For this section, you'll submit: perceiver.finetune.params, perceiver.pretrain.dev.predictions, and perceiver.pretrain.test.predictions. Your model should get at least 6% accuracy on the dev set.

- i. (8 points) We'll score your model as to whether it gets at least 5% accuracy on the test set, which has answers held out.
- ii. (2 points) Provide an expression for the time complexity of the Perceiver model and the vanilla model, in terms of number of layers (L), input sequence length (ℓ) and basis bottleneck dimension (m).

(i) The accuracy of the model on the dev set: **6.8%**

$$\begin{aligned}
 \text{(ii)} \quad TC_{\text{vanilla}} &= \left(\left(\left(l \times \frac{d}{h} \right)_{XQ_i} + \left(l \times \frac{d}{h} \right)_{XK_i} + (l \times l)_{XQ_i(XK_i)^T} + \left(l \times \frac{d}{h} \right)_{XV_i} + \left(l \times \frac{d}{h} \right)_{XQ_i(XK_i)^T XV_i} \right) \times h \right)_{\text{Single Layer}} \times L \\
 &= (4ld + l^2h)L \approx l^2Lh
 \end{aligned}$$

$$\begin{aligned}
TC_{\text{Perceiver}} &= \left(\left(\left(m \times \frac{d}{h} \right)_{CQ_i} + \left(l \times \frac{d}{h} \right)_{XK_i} + (m \times l)_{CQ_i(XK_i)^T} + \left(l \times \frac{d}{h} \right)_{XV_i} + \left(m \times \frac{d}{h} \right)_{CQ_i(XK_i)^T XV_i} \right) \times h \right)_{\text{First Layer}} \\
&+ \left(\left(\left(l \times \frac{d}{h} \right)_{XQ_i} + \left(m \times \frac{d}{h} \right)_{YK_i} + (l \times m)_{XQ_i(YK_i)^T} + \left(m \times \frac{d}{h} \right)_{YV_i} + \left(l \times \frac{d}{h} \right)_{XQ_i(YK_i)^T YV_i} \right) \times h \right)_{\text{Last Layer}} \\
&+ \left(\left(\left(m \times \frac{d}{h} \right)_{YQ_i} + \left(m \times \frac{d}{h} \right)_{YK_i} + (m \times m)_{YQ_i(YK_i)^T} + \left(m \times \frac{d}{h} \right)_{YV_i} + \left(m \times \frac{d}{h} \right)_{YQ_i(YK_i)^T YV_i} \right) \times h \right)_{\text{Hidden Layers}} \times L \\
&= 4ld + 4md + 2mlh + (4md + m^2h)L \approx m^2Lh
\end{aligned}$$

3. Considerations in pretrained knowledge (5 points)

- (a) (1 point) Succinctly explain why the pretrained (vanilla) model was able to achieve an accuracy of above 10%, whereas the non-pretrained model was not.

Pretraining allows the model to be a language model, capturing the essence of sentences and the meaning of the underlying words. Fine tuning a language model is better than training a model from scratch.

- (b) (2 points) Take a look at some of the correct predictions of the pretrain+finetuned vanilla model, as well as some of the errors. We think you'll find that it's impossible to tell, just looking at the output, whether the model *retrieved* the correct birth place, or *made up* an incorrect birth place. Consider the implications of this for user-facing systems that involve pretrained NLP components. Come up with two **distinct** reasons why this model behavior (i.e. unable to tell whether it's retrieved or made up) may cause concern for such applications, and an example for each reason.

- 1- This could cause the spread of factual errors. For example, people use language models like ChatGPT in everything. Asking a language model about a fact and delivering false information instead of saying: "I don't know" spreads rumors.
- 2- Difficulty of evaluation as we should have a huge database for manually testing the predictions of the language model. For example, a language model that is used to caption images automatically could write any words not related to the image. We should evaluate them manually one by one.

(c) (2 points) If your model didn't see a person's name at pretraining time, and that person was not seen at fine-tuning time either, it is not possible for it to have "learned" where they lived. Yet, your model will produce *something* as a predicted birth place for that person's name if asked. Concisely describe a strategy your model might take for predicting a birth place for that person's name, and one reason why this should cause concern for the use of such applications.

(While 3b discussed the problems that could arise from made up predictions, 3c asks for a mechanism the model could be using for generating birth places of people not seen at fine-tuning time and why such a mechanism could be problematic.)

It could generate the same birthplace as someone whose name is very similar to the person's name. This means that the model correlates a person's name to his birthplace and that is incorrect.